# Deeper Networks, Built from Blocks

Advanced Machine Learning

Sam Ogden

# DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

# Learning objectives

- Explain what a receptive field is, and why stacked small convolutions can recognize large structures
- Build a deep network from a repeated block (VGG), and say what changes between blocks
- Compare stacked $3 \times 3$ convolutions with one larger kernel, in parameter count
- Describe a $1 \times 1$ convolution, and how global average pooling replaces a large linear layer
- Explain the degradation problem, and why a residual block ($\mathbf{y} = \mathbf{x} + f(\mathbf{x})$) fixes it
- Write a residual block as an `nn.Module`, including the shortcut
- Say what a trained CNN's layers represent, and why they are reusable

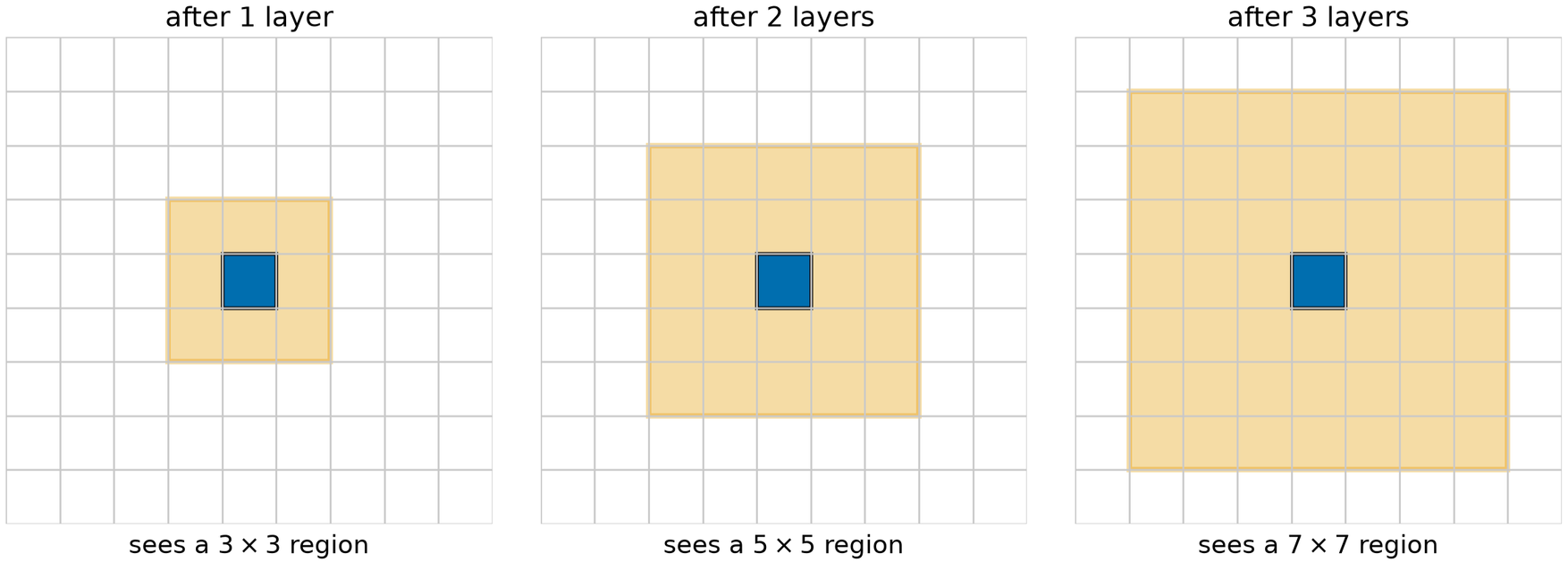
# Why go deeper

What can a $3 \times 3$ filter possibly see?

# A small filter, a big picture

- The CNN we built had two convolutional layers and read $28 \times 28$ gray digits
- A photograph is $224 \times 224$ and in color, and the thing we want to recognize (a dog, a face) can fill most of it
- But every filter we have looks at a $3 \times 3$ **patch**
- How can a model made of tiny windows recognize something large?

# What one output value can see




- The blue cell is one output value. The orange square is the part of the **original image** that can affect it
- Layer 2 looks at a $3 \times 3$ window of layer 1's outputs, and each of those looked at its own $3 \times 3$ window of the image

# The receptive field, with pooling

| Layer | Feature map ($C$ channels) | Region of the image one value sees |
|---|---|---|
| conv 3×3 | $C \times 224 \times 224$ | $3 \times 3$ |
| conv 3×3 | $C \times 224 \times 224$ | $5 \times 5$ |
| max pool 2×2 | $C \times 112 \times 112$ | $6 \times 6$ |
| conv 3×3 | $C \times 112 \times 112$ | $10 \times 10$ |
| conv 3×3 | $C \times 112 \times 112$ | $14 \times 14$ |
| max pool 2×2 | $C \times 56 \times 56$ | $16 \times 16$ |
| conv 3×3 | $C \times 56 \times 56$ | $24 \times 24$ |
| conv 3×3 | $C \times 56 \times 56$ | $32 \times 32$ |

- Each convolution adds a little. Each **pooling layer doubles how much every later layer adds**, because one cell now stands for a $2 \times 2$ block
- Eight small layers later, one value sees a $32 \times 32$ region

# From edges to objects

- Early layers see a tiny region, so all they can detect is **simple patterns**: edges, corners, color changes
- Middle layers combine those into **textures and parts**: a grid, a curve, an eye
- Late layers combine parts into **whole objects**: a face, a wheel, a dog
- Nobody tells the network to organize itself this way. The only requirement is that each layer reads the one before it, and training keeps whichever filters help the final answer

# Same recipe, bigger

## Same as our CNN

- Convolution, ReLU, max pooling, repeated
- A linear classifier on the flattened features at the end
- Cross-entropy loss, trained by gradient descent

## What the 2012 ImageNet winner (AlexNet) added

- 5 convolutional layers and 3 linear layers, for $224 \times 224$ color photos and 1,000 classes
- ReLU, instead of sigmoid, so a deeper stack still trains
- Dropout in the linear layers, and data augmentation
- GPUs, and 1.2 million training images

# Takeaway

**Stack many small convolutions: each layer looks at patterns of the layer before, so deeper layers see larger structures built out of smaller ones. The ingredients from last time (ReLU, dropout, augmentation, more data) are what made it possible to train.**

# Blocks: VGG

How do you design a network with 11 layers, or 100?

# Too many choices

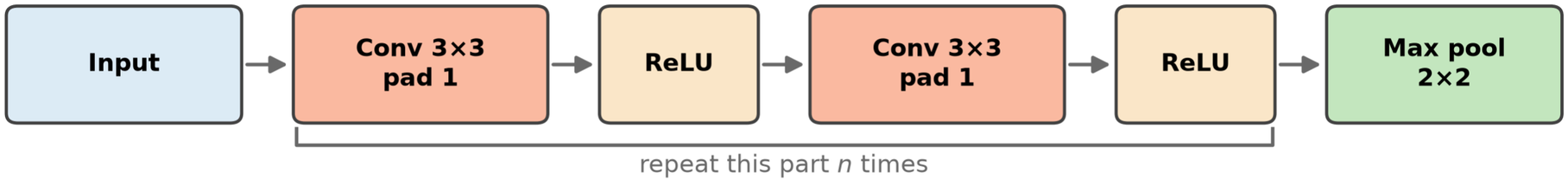
- Every layer needs a kernel size, a channel count, and a stride, and whether it pools
- With 10 layers that is about 40 decisions, and 100 layers is out of the question
- Trying them one at a time is hopeless, and they interact
- And even if we had the time, how would we **describe** the result to someone else?

> **Tip**
>
> These would all be **hyperparameters** that we'd be changing.
>
> Hyperparameters are parameters that aren't trained during SGD, but instead set prior to the training run.

# The Big Idea

**Design one small repeating block, then stack it. The network is then a short list: for each block, how many convolutions and how many channels.**

# The VGG block




- A few $3 \times 3$ convolutions, each followed by ReLU, then one max pool
- The convolutions **keep the height and width**: a $3 \times 3$ kernel with padding 1 gives $\lfloor (n + 2 - 3)/1 \rfloor + 1 = n$ for an $n \times n$ input. Only the channel count can change
- The max pool is the only layer that changes the size: it **halves** the height and width
- A block has two knobs: how many convolutions, and how many output channels

# Why $3 \times 3$? Same view, fewer weights

- To see a $5 \times 5$ region we have two options: **one** $5 \times 5$ convolution, or **two** $3 \times 3$ convolutions in a row
- Both give a value that depends on the same $5 \times 5$ patch of the image, as in the receptive-field picture
- So the question is which option needs fewer weights to learn

# Why $3 \times 3$? Counting the weights

- Say a layer takes $c$ channels in and gives $c$ channels out. Every (input channel, output channel) pair has its own kernel, so there are $c \cdot c = c^2$ kernels
- Each kernel has one weight per position: $3 \cdot 3 = 9$ for a $3 \times 3$ kernel, $5 \cdot 5 = 25$ for a $5 \times 5$ kernel (biases ignored)

$$\underbrace{2 \cdot (3 \cdot 3) \cdot c^2}_{\text{two } 3\times3 \text{ layers}} = 18c^2 \qquad \text{vs.} \qquad \underbrace{(5 \cdot 5) \cdot c^2}_{\text{one } 5\times5 \text{ layer}} = 25c^2$$

- The stack is cheaper, and it has a ReLU between its layers, so it can compute things a single layer cannot

# Why $3 \times 3$? With real numbers

| Same region seen | Stack of $3 \times 3$ | One larger kernel | Saved |
|---|---|---|---|
| $5 \times 5$ | 2 layers: 73,728 | 102,400 | 28% |
| $7 \times 7$ | 3 layers: 110,592 | 200,704 | 45% |

- With $c = 64$ channels. The gap widens as the region grows: **small kernels, stacked, are cheaper and more expressive**

# The block in PyTorch

```
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    return nn.Sequential(*layers)
```

- A function that **returns a network piece**: build it once, call it as many times as we like
- The whole network is then a list of `(num_convs, out_channels)` pairs

# A network from a list

```
def vgg(conv_arch, in_channels=3, num_classes=1000):
    blocks = []
    for num_convs, out_channels in conv_arch:
        blocks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels
    return nn.Sequential(
        *blocks,
        nn.Flatten(),
        nn.LazyLinear(4096), nn.ReLU(), nn.Dropout(0.5),
        nn.LazyLinear(4096), nn.ReLU(), nn.Dropout(0.5),
        nn.LazyLinear(num_classes)
    )

conv_arch = ((1, 64), (1, 128), (2, 256), (2, 512), (2, 512))     # VGG-11
```

- Five blocks (8 convolutions) then three linear layers: **11 weight layers**
- Changing the network means changing the list, not the code

# VGG in a picture

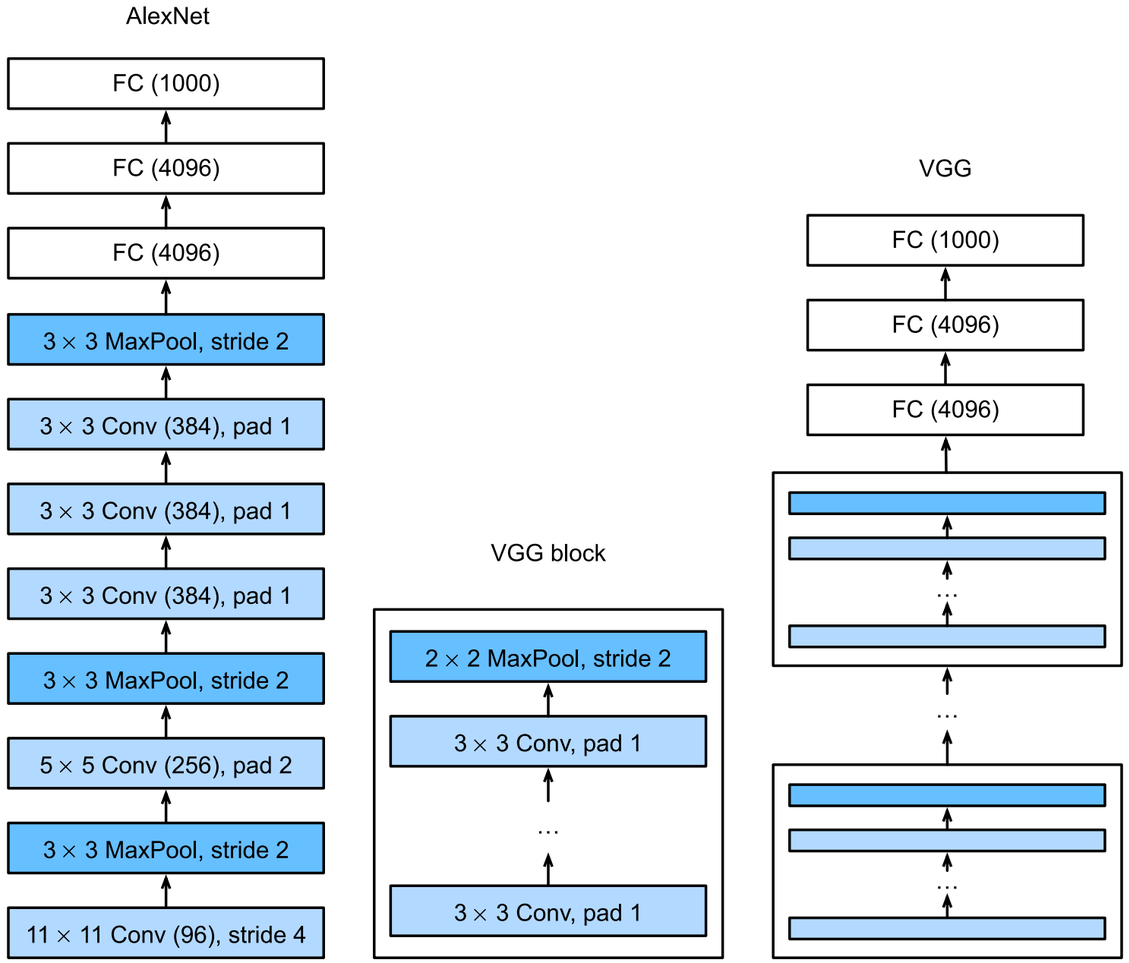



Figure: Zhang et al., *Dive into Deep Learning*, §8.2 (d2l.ai), CC BY-SA 4.0

# Tracing the shapes

| After | Output shape |
|---|---|
| input | `(1, 3, 224, 224)` |
| block 1 | `(1, 16, 112, 112)` |
| block 2 | `(1, 32, 56, 56)` |
| block 3 | `(1, 64, 28, 28)` |
| block 4 | `(1, 128, 14, 14)` |
| block 5 | `(1, 128, 7, 7)` |

- Shown with channels divided by 4 so it runs instantly. In full VGG-11 the channels are $64, 128, 256, 512, 512$

# Halve the size, grow the channels

- Every block **halves** the height and width ($224 \to 112 \to 56 \to 28 \to 14 \to 7$)
- Every block (up to a cap) **grows** the number of channels
- Early on: many positions, few kinds of pattern. Late: few positions, many kinds of pattern
- The picture shrinks and the **representation gets richer**: “where exactly” is traded for “what is here”

# Takeaway

**A block is the unit of design. Stacking blocks that halve the size and grow the channels is how we get depth without choosing every layer by hand.**

# Shrinking the classifier

# The parameters are in the wrong place

| Model | Total parameters | In the head | Share |
|---|---|---|---|
| AlexNet | 61,100,840 | 58,631,144 | 96% |
| VGG-11 | 132,863,336 | 123,642,856 | 93% |

- The **head** is the part of the network after the convolutional layers: the linear layers that turn their output into class scores
- In AlexNet and VGG, almost everything we pay for is in the head, not the convolutions
- VGG's first linear layer alone: $512 \times 7 \times 7 = 25{,}088$ features in, $4096$ out, so $102{,}760{,}448$ weights
- Those weights cost memory and time, and give the model plenty of room to overfit. Do we need a layer this big?

# The Big Idea

**The convolutional layers already did the hard work. Instead of a giant linear layer to read their output, mix the channels at each position, then average each channel over all positions.**

# The $1 \times 1$ convolution

- A kernel of size $1 \times 1$ looks at **one position** and **all the channels** at that position
- It is a linear layer applied **independently at every pixel**, mixing the channels and ignoring neighbors
- Shape: $c_{in} \times H \times W \to c_{out} \times H \times W$. The height and width do not change
- Parameters: $c_{in} \cdot c_{out}$ (plus $c_{out}$ biases), no matter how big the picture is

# A $1 \times 1$ convolution, by hand

- One position with $c_{in} = 3$ channels: $(2, 1, 3)$. Two output channels, one weight vector each: $(1, 0, -1)$ and $(0.5, 0.5, 0.5)$

$$o_1 = 2 \cdot 1 + 1 \cdot 0 + 3 \cdot (-1) = -1$$

$$o_2 = 2 \cdot 0.5 + 1 \cdot 0.5 + 3 \cdot 0.5 = 3$$

- The same six weights are applied at **every** position of the feature map. That is weight sharing, again, only across channels instead of across space

# Global average pooling

- Take each channel's $H \times W$ feature map and replace it with its **average**. One number per channel
- Example, two channels of a $2 \times 2$ feature map:

$$\begin{matrix} 1 & 3 \\ 5 & 7 \end{matrix} \quad \rightarrow \quad \frac{1+3+5+7}{4} = 4$$

$$\begin{matrix} 0 & 2 \\ 2 & 0 \end{matrix} \quad \rightarrow \quad \frac{0+2+2+0}{4} = 1$$

- A $2 \times 2 \times 2$ block of numbers becomes just $(4, 1)$: one number answering "**how much of this pattern is there, anywhere?**"
- No parameters at all, and it works for any image size

# The new head, step by step

Start from the same place as before: the last block hands us a $512 \times 7 \times 7$ feature map

1. $1 \times 1$ **convolution to 1000 channels.** At each of the $7 \times 7$ positions, mix the 512 channels into 1000 numbers. The output is $1000 \times 7 \times 7$: one map per class, showing how strongly that class is indicated at each position
2. **Global average pooling.** Average each of the 1000 maps over its positions. The output is 1000 numbers: **one score per class**, and these are the scores the loss function takes
3. That is the whole head: no flatten and no big linear layers. The only weights are the $1 \times 1$ convolution's: $512 \cdot 1000 + 1000 = 513{,}000$

# The new head in PyTorch

```
head = nn.Sequential(
    nn.Conv2d(512, 1000, kernel_size=1),    # mix channels at each position
    nn.AdaptiveAvgPool2d(1),                # average each map over all positions
    nn.Flatten(),                           # (1000, 1, 1) -> 1000 scores
)
```

| Input | Output | Parameters |
|---|---|---|
| `(1, 512, 7, 7)` | `(1, 1000)` | 513,000 |
| `(1, 512, 14, 14)` (a larger image) | `(1, 1000)` | 513,000 |

- A bigger feature map gives the same 1000 scores from the same weights, because the average does not care how many positions there are

# Old head vs. new head

## Flatten, then big linear layers

- $512 \times 7 \times 7 = 25{,}088$ numbers, one per channel *and position*
- Linear to 4096: $\approx 102.8$M weights
- Tied to exactly this image size

## $1 \times 1$ convolution, then global average pool

- Classify at each position, then average: $512 \cdot 1000 + 1000 = 513{,}000$ weights
- Works for any image size
- Averaging first and then applying one linear layer gives the same scores, and many networks are built that way

# Where this head comes from: NiN

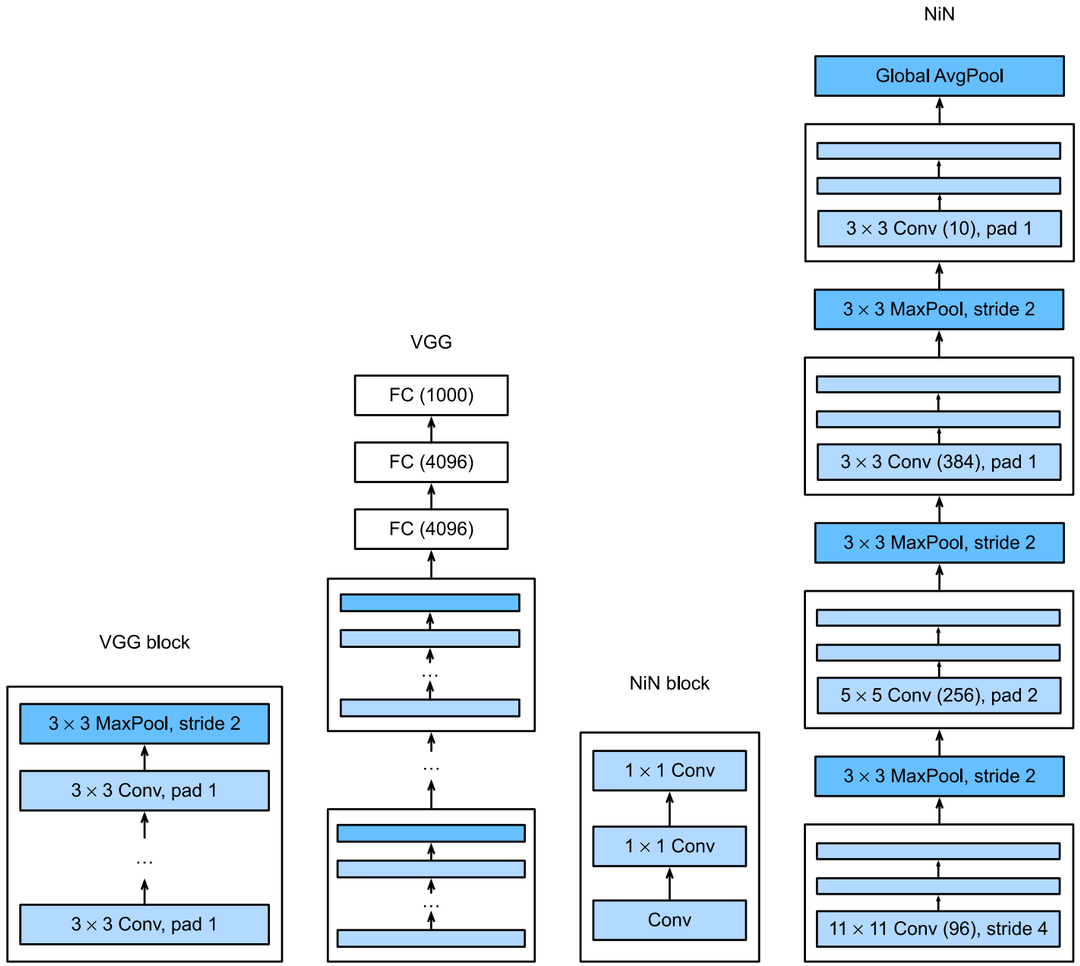



Figure: Zhang et al., *Dive into Deep Learning*, §8.3 (d2l.ai), CC BY-SA 4.0 (NiN: *Network in Network*, Lin et al., 2013)

# Takeaway

**A $1 \times 1$ convolution gives us a learned way to combine channels at each position, so we can then average over positions. The huge linear layer needed a weight for every position, and now we do not.**

# Residual networks

Can we just keep stacking?

# More layers should never hurt

- A deeper network can do everything a shallower one can: let the extra layers pass their input through unchanged, and we have the shallow network back
- So adding layers should never make the best possible result worse
- In practice, past a certain depth, **it does**
- Add layers to a plain stack and the model gets worse. Where does the problem come from?

# The degradation problem

- In the original ResNet paper (He et al., 2015), a 56-layer plain network did **worse than a 20-layer one** on CIFAR-10
- Not just on the validation set. On the **training set** too
- Batch norm and ReLU were already in use, which tame the worst of the vanishing-gradient problem. The deeper stack was **still** harder to train
- The extra layers could, in principle, have copied the 20-layer solution. The optimizer **failed to find it**

# Not overfitting

Overfitting: training error is low, validation error is high.
Degradation: **training** error is high.

- A bigger model that fits the training data *worse* is not memorizing too much. It is **not being trained well enough**
- From the curve shapes last time: this is a “both curves are bad” problem, not a gap problem
- So the fix is not more data or dropout. It has to change **how the layers are connected**

# Gradients shrink on the way back

- One reason a deep stack is hard to train: backprop reaches an early layer by multiplying one factor per layer on the way back, so the gradient there is a **product** of many numbers
- If each factor is below 1, the product shrinks fast. A sigmoid's slope is at most $0.25$, and over 8 layers $0.25^8 \approx 0.000015$
- Early layers then get almost no signal and learn very slowly. This is the **vanishing gradient** problem
- ReLU and batch norm help, but a very deep stack is still hard to train. We want the gradient to have a way **past** the layers

# The Big Idea

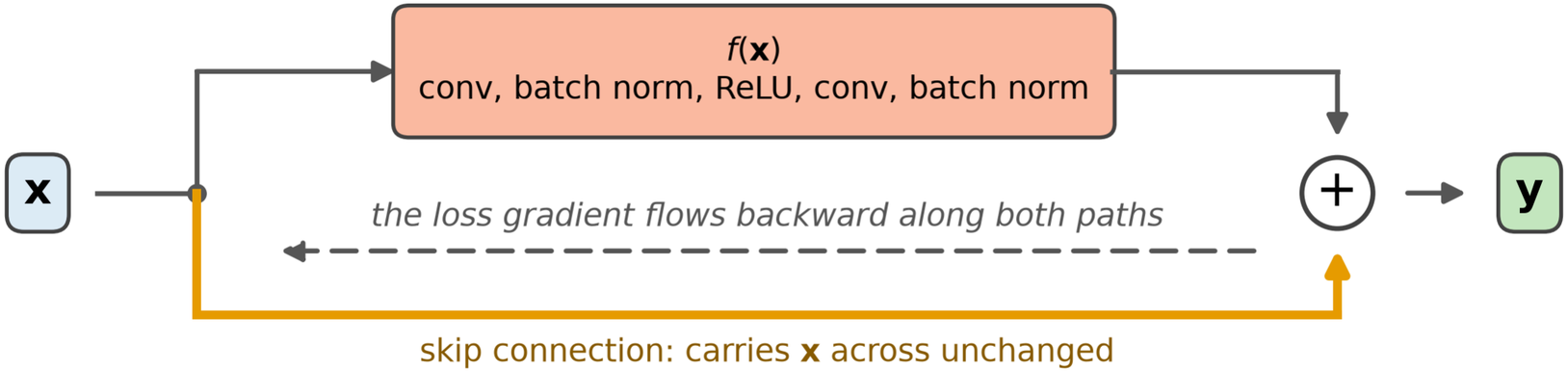
**Add each block’s input to its output. The gradient gets a direct path around the block, and the block only has to learn a correction.**

# The residual block

$$\mathbf{y} = \mathbf{x} + f(\mathbf{x})$$

- $\mathbf{x}$ is the block's input, $\mathbf{y}$ its output
- $f$ is a few layers of convolution, batch norm and ReLU, the same kind of thing we have been stacking
- The input is **added** to the output of $f$, so the two must have the same shape
- The path that carries $\mathbf{x}$ straight across is the **skip connection** (or shortcut)

# Breakdown: the skip connection




- The skip path has **no parameters and no computation**: it just copies
- Whatever else the block does, the input is still there at the output

# Breakdown: the residual function

- $f$ only has to learn **what to add to $\mathbf{x}$**, not to rebuild $\mathbf{x}$ from scratch
- If the best thing for this block is to do nothing, $f$ only has to produce zeros. For a plain stack, the layers would have to learn the identity exactly
- So a block that is not needed can learn to do nothing: with $f \approx 0$, the network behaves as it did without the block

# One value through a block

- Say one entry of $\mathbf{x}$ is $3.0$ and the block's layers compute $f = 0.5$
- Plain layer: the output is whatever the layers say, $0.5$. The $3.0$ is gone
- Residual block: $y = 3.0 + 0.5 = 3.5$. The $3.0$ is kept, with a nudge
- If training pushes the layers' weights to zero, $f = 0$, so $y = 3.0$: **the identity, for free**

# Why gradients survive

- Differentiate $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$ with respect to $\mathbf{x}$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = 1 + \frac{\partial f}{\partial \mathbf{x}}$$

- The $1$ comes from the skip: the gradient has a direct path around the block, however small $\partial f / \partial \mathbf{x}$ is. Backprop multiplies one of these per block, going backwards
- Back to the vanishing-gradient picture, with a made-up factor of $0.1$ per layer (not measured). Over 8 layers:

**Plain stack**

$$0.1^8 = 10^{-8}$$

The gradient all but vanishes

**Residual blocks**

$$(1 + 0.1)^8 \approx 2.1$$

The gradient survives

# When the shapes do not match

- The sum $\mathbf{x} + f(\mathbf{x})$ needs both to have the same shape
- When a block changes the number of channels or halves the size (stride 2), the skip path cannot copy $\mathbf{x}$ as it is
- Fix: put a $1 \times 1$ **convolution** (with the same stride) on the skip path. It mixes the channels to the new count and skips positions to match
- Blocks that keep the shape use the plain copy. Only the blocks that change it pay for the extra convolution

# The two kinds of residual block

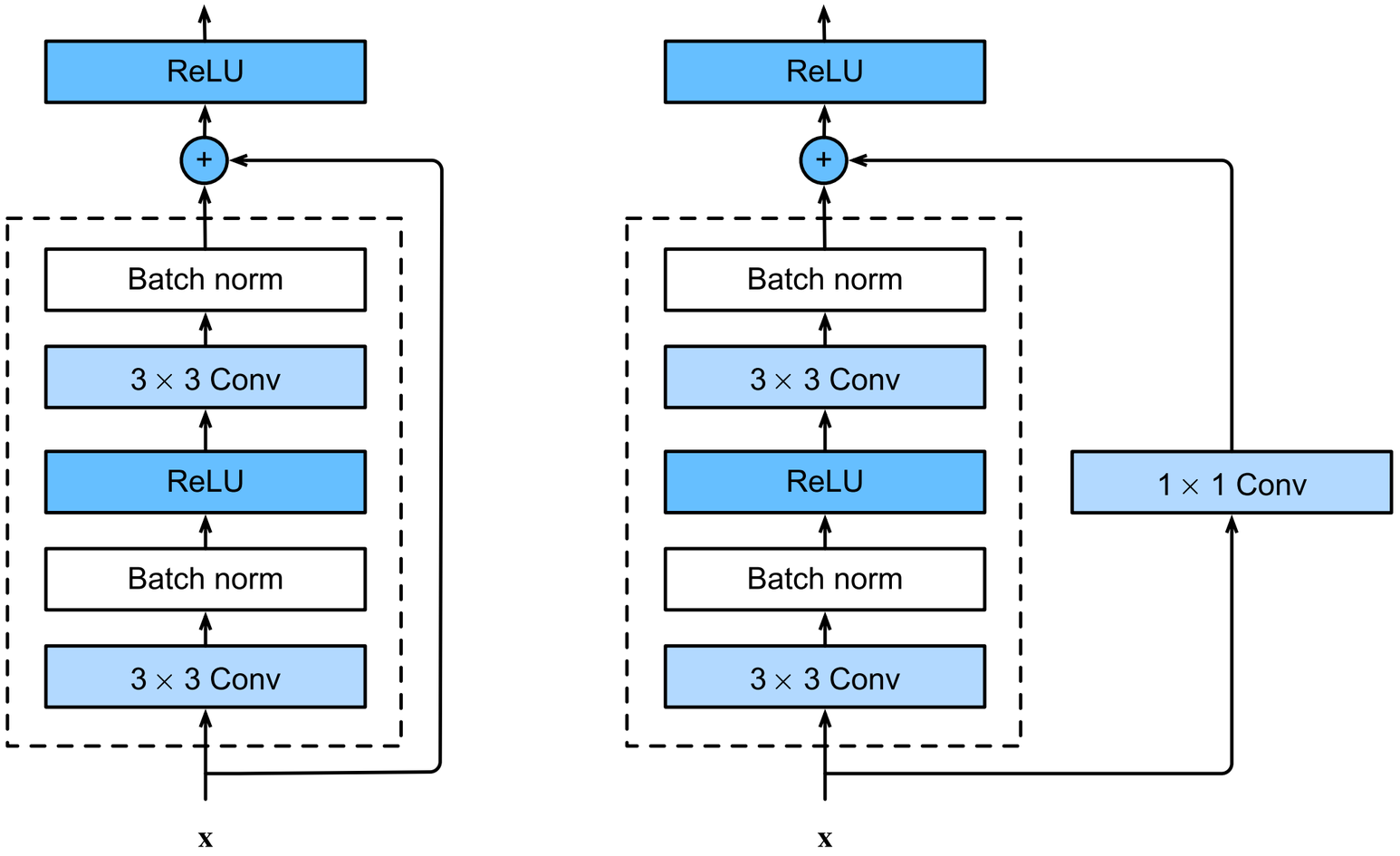



Figure: Zhang et al., *Dive into Deep Learning*, §8.6 (d2l.ai), CC BY-SA 4.0

# The block in PyTorch

```
 1 class Residual(nn.Module):
 2     def __init__(self, in_channels, out_channels, stride=1):
 3         super().__init__()
 4         self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride=stride, padding=1)
 5         self.bn1 = nn.BatchNorm2d(out_channels)
 6         self.conv2 = nn.Conv2d(out_channels, out_channels, 3, padding=1)
 7         self.bn2 = nn.BatchNorm2d(out_channels)
 8         self.skip = None
 9         if in_channels != out_channels or stride != 1:
10             self.skip = nn.Conv2d(in_channels, out_channels, 1, stride=stride)
11 
12     def forward(self, x):
13         out = F.relu(self.bn1(self.conv1(x)))
14         out = self.bn2(self.conv2(out))
15         shortcut = x if self.skip is None else self.skip(x)
16         return F.relu(out + shortcut)
```

- `forward` is the formula: `out` is $f(\mathbf{x})$, `shortcut` is $\mathbf{x}$, and the block returns their sum (then a ReLU)

# Shapes through a residual block

| Block | Input | Output | Skip path | Parameters |
|---|---|---|---|---|
| `Residual(64, 64)` | `(1, 64, 56, 56)` | `(1, 64, 56, 56)` | copy | 74,112 |
| `Residual(64, 128, stride=2)` | `(1, 64, 56, 56)` | `(1, 128, 28, 28)` | $1 \times 1$ conv | 230,272 |

# ResNet-18

| Part | What it is | Output shape |
|---|---|---|
| Stem | $7 \times 7$ conv, $3 \times 3$ max pool | $64 \times 56 \times 56$ |
| Stage 1 | 2 blocks, 64 channels | $64 \times 56 \times 56$ |
| Stage 2 | 2 blocks, 128 channels, stride 2 | $128 \times 28 \times 28$ |
| Stage 3 | 2 blocks, 256 channels, stride 2 | $256 \times 14 \times 14$ |
| Stage 4 | 2 blocks, 512 channels, stride 2 | $512 \times 7 \times 7$ |
| Head | Global average pool, linear | $1000$ |

- In stages 2 to 4, the first block halves the size and doubles the channels, and it is the one with the $1 \times 1$ skip. 18 weight layers: 1 in the stem, 16 in the blocks, 1 in the head
- About 11.7M parameters, roughly one-eleventh of VGG-11's, and at least as accurate on ImageNet

# Takeaway

**A residual block adds its input back to its output, so every block starts as “do nothing” and learns a correction. That is what makes networks with dozens, and even hundreds, of layers trainable.**

# What the layers learned

Reusing the representation

# The head is the same linear classifier

- A ResNet is a stack of blocks that turns a photograph into **512 numbers** (after the global average pool)
- The final layer, `Linear(512, 1000)`, is the same linear classifier we have used since the first classification lecture
- All the work of making classes **linearly separable** happened in the blocks before it
- Nothing about those 512 numbers is specific to the 1,000 classes the network was trained on. They describe how much of each learned pattern is present

# A trained network is a feature extractor

- Take a network trained on a huge dataset, keep every block, and throw away only the last linear layer
- What is left maps **any** image to a vector of features that describes it
- Train a new, small linear classifier on those features for **our own** problem, with our own classes and far less data
- That is **transfer learning**, which is where we go next

# Takeaway

**Deep networks learn representations in layers, from edges to objects. Those representations are not only the means to one classification: they are reusable.**

# Summary

# Recap: what each design contributes

| Idea | What it does | Why it matters |
|---|---|---|
| Stacked small kernels | Receptive field grows layer by layer | Large structures from small filters, fewer parameters |
| The block (VGG) | One repeated unit; halve the size, grow the channels | Describe a deep net in a list |
| $1 \times 1$ convolution | Mix channels at each position | Reshape channel counts cheaply |
| Global average pooling | One number per channel | Replaces a huge linear head |
| Residual connection | $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$ | Very deep networks train |

# Recap: key ideas

- Each layer reads the one before it, so **deeper layers see larger structures**: edges, then parts, then objects
- Design a **block** and repeat it, rather than choosing every layer
- Most parameters can hide in the linear head. $1 \times 1$ **convolutions and global average pooling** remove it
- A plain stack gets **harder to train** with depth, even on the training set. That is not overfitting
- **Residual blocks** keep the input and learn a correction, so the gradient always has a direct path

# Taking one idea to the extreme

**Each of these networks took one idea as far as it would go: stack small convolutions (VGG), use nothing but convolutions (NiN), add the input back at every block (ResNet).**

# Where residuals show up next

- The residual connection is not just a CNN trick. It appears again, almost unchanged, in the **Transformer** blocks at the heart of language models
- Whenever we meet a very deep stack, expect an $\mathbf{x} + f(\mathbf{x})$ inside it
- Next time: using a trained network's representations for a new problem, and looking at where such a model goes wrong